

ClubDash AI Assistant

Implementation & Technical Documentation Report

Member + Front-Desk Staff • Phase 1 Read-Only Conversational Assistant

This document records the chatbot feature implemented for the ClubDash sports-club management platform. It explains the purpose, architecture, frontend, backend API, role-based tools, security model, setup, testing approach, and future development plan.

Implementation date: 10 August 2026

1. Executive Summary

ClubDash now includes a role-aware AI assistant for authenticated Members and Front-Desk Staff. The assistant is presented as a floating chat widget so users can request help without leaving the page they are using. The first release is intentionally read-only: it retrieves live ClubDash information but does not modify bookings, attendance, events, memberships, courts, or clubs.

The key engineering idea is controlled tool calling. The AI model does not receive direct database access. Instead, the backend exposes a restricted set of server-side functions. When a question requires live data, the model selects an approved function, the Flask backend executes that function using existing ClubDash services/models, and the result is returned to the model to formulate the final response.

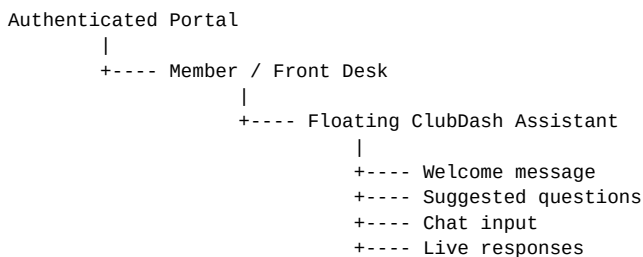
2. Why the Chatbot Was Added

- Reduce the need for members and staff to navigate multiple screens for simple questions.
- Provide contextual help while the user is already inside the Member or Staff portal.
- Allow natural-language access to existing ClubDash information.
- Demonstrate AI integration without bypassing the existing REST/API and authorization architecture.
- Create a foundation for future action-based capabilities such as booking, cancellation and staff check-in.

3. User Experience Design

3.1 Placement

The assistant is not initially a separate navigation page. It is a floating button at the bottom-right of authenticated Member and Front-Desk screens. Clicking the button opens a compact chat panel.



3.2 Member experience

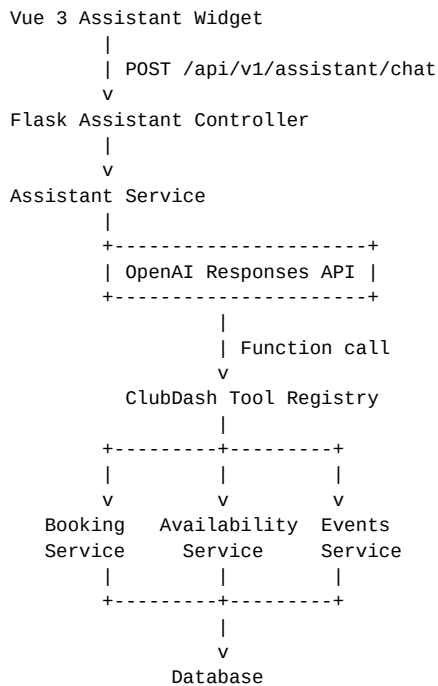
- Ask for upcoming personal bookings.
- Ask about upcoming events.
- Ask which courts are available.
- Ask about current membership information.
- Ask for club address and operating hours.

3.3 Staff experience

- Ask for today's booking situation.
- Ask about court status.
- Ask who is currently checked in.
- Search staff booking information.

- Ask for an operational dashboard summary.

4. System Architecture



The assistant is an additional application layer over the existing backend. It does not replace the booking, availability, events or attendance services. This preserves the existing separation between controllers, services and models.

5. Backend Implementation

5.1 New backend module

```

backend/app/assistant/
  __init__.py
  controllers.py
  services.py
  tools.py
  
```

5.2 Controller

The controller exposes POST /api/v1/assistant/chat. It requires the existing JWT authentication cookie, identifies the current User, validates the message, and delegates processing to AssistantService.

```

POST /api/v1/assistant/chat

Request:
{
  "message": "What are my upcoming bookings?"
}

Response:
{
  "assistant_message": "...",
  "response_id": "resp...",
  "tools_used": ["get_my_bookings"]
}
  
```

5.3 Assistant service

- Creates the server-side OpenAI client using OPENAI_API_KEY.
- Uses a role-specific instruction set.

- Provides only the tools allowed for the authenticated role.
- Processes model function calls.
- Executes the selected ClubDash tool on the server.
- Sends the tool result back to the model.
- Returns the final natural-language response.
- Hides provider errors and internal implementation details from the browser.

6. Role-Based Tool Architecture

Tool	Member	Staff	Purpose
get_my_bookings	Yes	Yes	Authenticated user's bookings
get_events	Yes	Yes	Upcoming events + registration state
get_availability	Yes	Yes	Live court availability
get_my_membership	Yes	Yes	Membership/plan information
get_club_information	Yes	Yes	Club details and hours
get_staff_dashboard	No	Yes	Today's operational summary
get_staff_bookings	No	Yes	Staff booking search
get_staff_attendance	No	Yes	Attendance records
get_court_status	No	Yes	Court availability summary

7. Member Tool Details

7.1 My bookings

Reads Booking records belonging to the authenticated user. The assistant serializes date, time, status, court and club.

7.2 Events

Reads upcoming Event records and calculates registration count and the authenticated member's own registration status.

7.3 Availability

Reuses AvailabilityService.get_availability_matrix(), so the assistant reports the same booking availability used by the existing UI.

7.4 Membership

Reads Membership records and associated MembershipPlan information for the authenticated user.

7.5 Club information

Returns club name, address, opening and closing time, slot duration and active court count.

8. Staff Tool Details

8.1 Staff dashboard

Reuses `StaffService.dashboard()` and reports total/active courts, today's bookings, active bookings, checked-in members and pending arrivals.

8.2 Staff bookings

Supports date, status and search filtering across member name, member email and court name.

8.3 Staff attendance

Returns attendance records for the staff member's club, defaulting to today's records.

8.4 Court status

Uses the existing availability matrix and summarizes each court into available, booked and blocked slot counts.

9. Security and Authorization

- The endpoint requires JWT authentication.
- The current user is resolved from the authenticated JWT identity.
- Member and Staff tool lists are different.
- Owner accounts are not enabled for the assistant in Phase 1.
- The browser cannot grant itself staff tool access because tool authorization is performed in the backend.
- The OpenAI API key is never sent to Vue.
- The previous response ID is stored server-side in the Flask session instead of being trusted from the browser.
- The assistant is explicitly instructed to remain read-only in Phase 1.

10. Read-Only Safety Boundary

Phase 1: READ ONLY

Allowed:

- ✓ Read bookings
- ✓ Read availability
- ✓ Read events
- ✓ Read membership
- ✓ Read attendance
- ✓ Read club information

Not allowed:

- ✗ Create booking
- ✗ Cancel booking
- ✗ Reschedule booking
- ✗ Register for event
- ✗ Check in member
- ✗ Check out member
- ✗ Edit membership
- ✗ Edit club/court

This boundary is deliberate. It lets the team validate AI responses and role separation before allowing an AI model to perform state-changing operations.

11. Frontend Implementation

11.1 New component

`Frontend/src/components/assistant/ClubAssistant.vue`

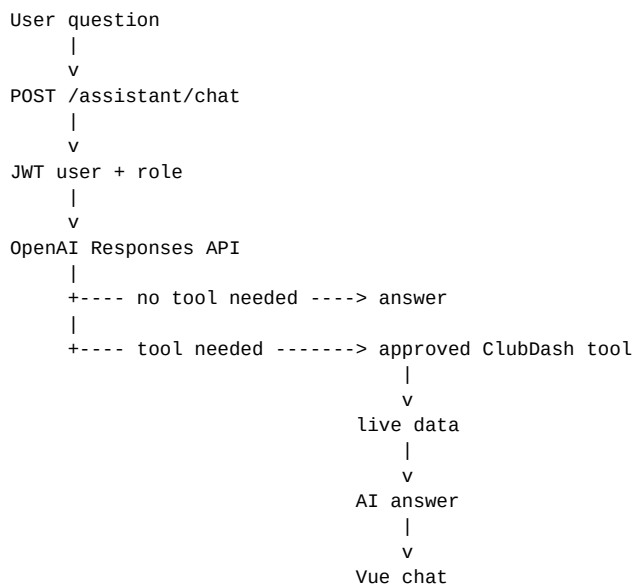
- Floating assistant button.

- Expandable chat panel.
- Role-specific welcome message.
- Role-specific suggested prompts.
- Scrollable conversation area.
- Loading indicator.
- Error handling.
- New conversation control.
- Responsive mobile layout.

11.2 Global mounting

App.vue mounts ClubAssistant globally. The component itself decides whether it should be visible based on `auth.user.role`.

12. AI Conversation Flow



The Responses API supports custom function tools and multi-turn response state. The implementation uses this capability to connect natural-language questions to controlled server-side ClubDash functions. The official OpenAI Python SDK documents the Responses API as the primary API and supports `client.responses.create()`, custom tools, and `previous_response_id` for multi-turn responses.

13. Configuration

backend/.env

```
OPENAI_API_KEY=your-openai-api-key
OPENAI_ASSISTANT_MODEL=gpt-5.5
```

The project includes `backend/.env.example`. The real `.env` file must remain private and must not be committed to Git.

14. Installation and Running

```
cd backend
pip install -r requirements.txt
python run.py
```

```
cd Frontend
npm install
npm run dev
```

The modified requirements add the OpenAI Python SDK. The frontend continues using the existing Axios client and HttpOnly JWT cookie.

15. API Documentation

The OpenAPI document has been extended with /assistant/chat so the new endpoint is visible through the project's Swagger UI.

16. Testing Plan

Test	Expected result
Unauthenticated chat	401 Unauthorized
Empty message	400 Validation Error
Message over 2000 characters	400 Validation Error
Member asks own bookings	Member booking tool executes
Member asks staff attendance	Staff tool is denied
Staff asks today's bookings	Staff booking tool executes
Owner calls assistant	403 Forbidden
No OPENAI_API_KEY	503 configuration response
Live availability question	Existing availability service is used
Write request in Phase 1	No write tool is called

17. Future Development

- Phase 2: confirmation-based booking, cancellation and rescheduling tools.
- Phase 2: event registration through the assistant.
- Phase 2: staff check-in and check-out tools.
- Phase 3: persistent assistant conversations and message history.
- Phase 3: owner/admin assistant with analytics and reporting tools.
- Phase 4: map/geolocation integration so the assistant can help members find nearby clubs.

18. Engineering Rationale

The design intentionally treats the chatbot as an interface layer rather than a replacement for the application backend. Existing business services remain the source of truth. This reduces duplicated logic, keeps authorization on the server, and makes the AI feature modular. It also means the same booking and availability rules are used whether the user interacts through the normal Vue pages or through the assistant.

19. Files Changed

File	Change
backend/app/assistant/controllers.py	New assistant API controller
backend/app/assistant/services.py	New AI orchestration service
backend/app/assistant/tools.py	New role-aware ClubDash tool registry
backend/app/assistant/__init__.py	New assistant module
backend/app/__init__.py	Registers assistant blueprint
backend/app/auth/controllers.py	Clears assistant response state on logout
backend/requirements.txt	Adds OpenAI SDK
backend/.env.example	Documents AI configuration
backend/docs/api-docs/openapi.yaml	Documents /assistant/chat
Frontend/src/components/assistant/ClubAssistant.vue	New assistant UI
Frontend/src/App.vue	Mounts assistant globally

20. Final Status

Phase 1 of the ClubDash AI Assistant has been implemented as a frontend + Flask backend feature with role-aware, read-only tool integration. Backend Python files were syntax-compiled successfully and the OpenAPI YAML was parsed successfully. A frontend production build was attempted in the provided project environment, but the archived node_modules contained an incompatible/missing native Rolldown binding; therefore the build could not be completed in this environment. Running npm install in the user's local Frontend directory should restore the platform-specific dependency before npm run build.